GUIA TÉCNICO

SQL avançado e otimização

Janelas, índices, desempenho e planos de execução

Propósito — Material de referência complementar aos notebooks. Exemplos locais com dados sintéticos; laboratórios externos somente sob configuração explícita.

Como utilizar — Leia as explicações, execute os exemplos no ambiente indicado e consulte os notebooks correspondentes.

ROTEIRO DO DOCUMENTO

1. 04 | SQL avançado e otimização
2. Objetivo e escopo
3. Conceito central: Plano de execução
4. Aplicação, ressalvas e limites
5. CTE e consulta analítica
6. Funções de janela
7. Média móvel de três observações
8. Índices e plano de execução

SQL avançado e otimização

Janelas, índices, desempenho e planos de execução

MAPA DE ESTUDO

01 Consultas analíticas e funções de janela

02 Índices e seletividade

03 Planos de execução e medição

04 Transações e operações seguras

05 Decisões de otimização e armadilhas

Material de referência complementar aos notebooks. Exemplos locais com dados sintéticos; laboratórios externos somente sob configuração explícita.

04 | SQL avançado e otimização

GUIA TÉCNICO APROFUNDADO • ANÁLISE, CÓDIGO E INTERPRETAÇÃO

Objetivo e escopo

Estudo independente baseado no notebook aprovado. Os exemplos locais usam SQLite e dados sintéticos; PostgreSQL, MongoDB e Cassandra requerem serviços de laboratório para verificação real.

Conceito central: Plano de execução

CTEs organizam consultas, mas não garantem ganho automático de velocidade. Funções de janela produzem contexto sem colapsar linhas; ROW_NUMBER exige critério estável de desempate. Índices aceleram acessos seletivos, mas oneram escrita e espaço.

Aplicação, ressalvas e limites

Compare EXPLAIN QUERY PLAN antes e depois do índice, mantendo os mesmos dados. Um plano diferente não comprova melhoria de tempo; meça também se a base justificar.

CTE e consulta analítica

CTEs ajudam a nomear etapas lógicas; não garantem desempenho superior por si mesmas.

Exemplo do notebook — execução local

```
import sqlite3
conn = sqlite3.connect(':memory:')
conn.execute('PRAGMA foreign_keys=ON')
conn.executescript((Path('sql')/'01_schema_sqlite.sql').read_text(encoding='
    utf8'))
leituras = pd.read_csv(DATA/'leituras_sensores.csv')
leituras.to_sql('leituras',conn,if_exists='append',index=False)
print('SQLite inicializado:',conn.execute('SELECT COUNT(*) FROM leituras').f
    etchone()[0], 'leituras')
```

Funções de janela

ROW_NUMBER e médias móveis preservam cada linha e acrescentam contexto por partição. GROUP BY reduziria a quantidade de linhas.

Exemplo do notebook — execução local

```
q='''WITH ranked AS (SELECT id_sensor,data_hora,temperatura_c, ROW_NUMBER()
    OVER(PARTITION BY id_sensor ORDER BY temperatura_c DESC) pos FROM leitur
    as) SELECT * FROM ranked WHERE pos<=2 ORDER BY id_sensor,pos''';display(
    pd.read_sql_query(q,conn))
```

Média móvel de três observações

O quadro inclui a observação corrente e duas anteriores por sensor; nas duas primeiras, a janela é menor.

Exemplo do notebook — execução local

```
display(pd.read_sql_query('''SELECT id_sensor,data_hora,temperatura_c, ROUND
```

```
    (AVG(temperatura_c) OVER (PARTITION BY id_sensor ORDER BY data_hora ROWS
     BETWEEN 2 PRECEDING AND CURRENT ROW),2) AS mm3 FROM leituras ORDER BY i
    d_sensor,data_hora LIMIT 9''',conn))
```

Índices e plano de execução

EXPLAIN QUERY PLAN mostra a estratégia escolhida pelo SQLite; um índice não é sempre benéfico e deve responder a consultas reais.

Exemplo do notebook — execução local

```
query='SELECT * FROM leituras WHERE id_sensor=2 ORDER BY data_hora'; print('
    Antes:');display(pd.read_sql_query('EXPLAIN QUERY PLAN '+query,conn));co
    nn.execute('CREATE INDEX IF NOT EXISTS idx_leituras_sensor_data ON leitu
    ras(id_sensor,data_hora)');print('Depois:');display(pd.read_sql_query('E
    XPLAIN QUERY PLAN '+query,conn))
```

Exercício resolvido - último valor por sensor

Usar ROW_NUMBER com data decrescente e isolar pos=1 para cada sensor. Se houver empates de timestamp, adicionar critério determinístico.

Exemplo do notebook — execução local

```
display(pd.read_sql_query('''WITH ult AS (SELECT id_sensor,data_hora,tempera
    tura_c,ROW_NUMBER() OVER(PARTITION BY id_sensor ORDER BY data_hora DESC,
    id_evento DESC) rn FROM leituras) SELECT id_sensor,data_hora,temperatura
    _c FROM ult WHERE rn=1 ORDER BY id_sensor''',conn))
```

--- ### Aprofundamento v2 - Índices e plano de consulta

Use EXPLAIN QUERY PLAN para verificar a escolha do otimizador. Um índice pode acelerar filtros seletivos, mas a vantagem depende de cardinalidade, distribuição, atualização e tamanho da tabela. Não confunda plano de execução com prova de ganho de desempenho; benchmarks precisam de cargas comparáveis e várias medições.

Interpretação: compare o resultado com a teoria anterior; este exemplo local não substitui um laboratório de serviço externo.

Exemplo do notebook — execução local

```
import sqlite3
with sqlite3.connect(':memory:') as db:
    db.execute('CREATE TABLE eventos(id INTEGER PRIMARY KEY, sensor INTEGER,
     valor REAL)')
    db.executemany('INSERT INTO eventos VALUES (?,?,?)',[(i,i%20,float(i%7))
     for i in range(1000)])
    antes=db.execute('EXPLAIN QUERY PLAN SELECT * FROM eventos WHERE sensor=
    7').fetchall()
    db.execute('CREATE INDEX idx_eventos_sensor ON eventos(sensor)')
    depois=db.execute('EXPLAIN QUERY PLAN SELECT * FROM eventos WHERE sensor
    =7').fetchall()
    print('Sem índice:',antes)
    print('Com índice:',depois)
    assert sum(1 for x in db.execute('SELECT * FROM eventos WHERE sensor=7')
    )==50
```

Exercício de revisão

Exercício resolvido - revisão: identifique a principal hipótese do exemplo acima, explique por que ela importa e cite uma condição na qual a técnica falharia. Resposta orientadora: registre a pré-condição (tipo de dado, chave, restrição, volume ou topologia), teste-a explicitamente e explique os limites dos resultados observados.

Aprofundamento: decisões e estudo de caso

Funções de janela

O uso de ROW_NUMBER permite classificar observações dentro de cada partição sem suprimir o conjunto original. A ordenação deve ser estável: se duas leituras tiverem exatamente o mesmo instante, inclua id_evento como critério de desempate. Uma janela ROWS BETWEEN 2 PRECEDING AND CURRENT ROW calcula a média das últimas três observações disponíveis, não necessariamente de três minutos.

Índices compostos

Em um índice (id_sensor, data_hora), a ordem das colunas é uma decisão. Consultas filtradas por sensor e ordenadas por data podem se beneficiar dele; consultas sem filtro por sensor podem não aproveitar a mesma estrutura da mesma forma. O índice exige espaço adicional e manutenção em INSERT/UPDATE/DELETE. Avalie seletividade e planos para o workload específico.

Planejamento e benchmark

EXPLAIN QUERY PLAN apresenta uma estratégia de acesso do SQLite, mas não é medição direta de latência. Um plano que mostra SEARCH em vez de SCAN pode sugerir uso de índice, porém a relevância depende da distribuição de dados e da quantidade de linhas. Em bases pequenas, uma varredura completa pode ser tão rápida quanto uma busca indexada.

Exercício de otimização

Escreva uma consulta para a leitura mais recente por sensor com ROW_NUMBER e outra com junção a subconsulta agregada. Garanta um desempate determinístico. Depois crie e remova um índice composto, registre os planos e descreva diferenças sem concluir ganho de desempenho apenas pela aparência do plano.

Checklist de interpretação

- Defina a pergunta, unidade de análise e pré-condições da consulta.
- Registre chaves, cardinalidade, restrições e tratamento de valores ausentes.
- Execute os exemplos locais antes de habilitar serviços reais.
- Separe evidência observada de hipóteses e propriedades não testadas.

Leituras e arquivos de apoio

Consulte o notebook correspondente, os scripts em sql/, os datasets sintéticos e a documentação oficial dos SGBDs. Os resultados locais não substituem testes de integração com serviços externos.